

FM Demodulator

Gage Shaddock

Overview

This project implements a real-time FM radio demodulator on the NVIDIA Jetson Orin Nano using CUDA kernels. An RTL-SDR V4 USB dongle captures raw IQ (in-phase/quadrature) samples from the air at 2.048 Msps, and a six-stage CUDA kernel pipeline converts those samples into demodulated 48 kHz mono audio. The pipeline runs fast enough to process incoming radio in real-time, with the GPU finishing each 131,072-sample block well before the next one arrives from the dongle.

Three GPU implementations were built and benchmarked against a sequential CPU reference: a naive GPU baseline, an optimized GPU version with shared-memory tiling and parallel DC reduction, and a multi-channel GPU version capable of demodulating up to 5 FM stations simultaneously using concurrent CUDA streams.

The Jetson's unified memory architecture, where CPU and GPU share the same physical memory, is exploited throughout the project. The RTL-SDR callback writes directly into a `cudaHostAllocMapped` buffer that the GPU can access without any explicit `cudaMemcpy`, eliminating transfer overhead entirely.

GPU Acceleration

The project pipeline is broken down into six stages that are all executed by cuda kernels. The first stage normalizes the `uint8` input to a float within the range of -1 to 1. In this stage there is one kernel for every IQ pair allowing for GPU parallelism. The second stage does a frequency shift operation on the new float data to shift the target frequency to 0Hz, this again uses one kernel for every IQ pair.

The next stage is a low pass filter that removes frequencies higher than 100kHz. This same stage also does a decimation by 7 to lower the number of samples by only keeping every seventh element. This stage is handled by a single kernel that spawns as many threads as there are output elements, thus parallelizing the filter operations and automatically decimating by seven. The next stage does the actual FM demodulation operations on each IQ sample, spawning one thread per decimated sample again leveraging the GPU's parallelism.

The next stage is separated into two kernels, a parallel reduction accumulator that runs one thread for every pair of IQ samples and a subtractor that removes this dc block from each

element in the output signal. The final stage is another low pass filter plus decimation step where much like the first one there is a thread for each output sample, thus saving time by only doing necessary calculations and using the parallel GPU structure.

Implementation

The project uses an Nvidia Jetson Orin Nano for its unified memory architecture between the CPU and GPU. This allows for the use of `cudaHostAlloc` and `cudaHostAllocMapped` to create memory pointers that can be used within the gpu kernels and the cpu functions without any copying overhead. This is specifically used to transfer input from the `rtl-sdr` call to the gpu pipeline and output from the gpu pipeline to the cpu for further processing. The project also uses a C struct to carry buffering data and pointers between the different gpu kernel calls.

The C host code only needs to interact with the create, destroy, or process functions which then use gpu cuda calls to set up or tear down memory buffers, transfer data, and call the appropriate kernels. The create function runs a warm up pass to initialize the gpu, then the `rtl` callback calls the process function after transferring data into the appropriate buffer. The process call then calls all kernel functions within a single cuda stream. The multi-channel version calls each kernel function for every channel, using one cuda stream per channel, to run the same pipeline on slightly different frequency offsets. This multiple streams implementation allows the gpu parallel benefits to shine, processing five different FM radio stations in real time.

The project is split into four implementations: a CPU sequential version, an initial unoptimized GPU version, an optimized GPU version, and a multi-channel GPU version. The CPU version is also built to run on the Jetson for benchmarking, providing a control group to compare the speedups of the GPU versions. The initial unoptimized GPU version launched more kernels to process I and Q data separately in many stages and did not use any shared memory or reduction strategies. The optimized version combined two of the initial stages into one and rewrote a few kernels to handle both I and Q calculations to reduce data transfer and kernel launching overhead. The optimized version also implemented parallel reduction on the dc removal stage and shared memory tiling in the fm demodulation and filtering / decimation stages. The multi channel version used multiple streams to call these same kernels at different frequency offsets from our tuned center, producing five different demodulated waveforms.

Documentation

Project requires librtlsdr github repo to setup the RTL-SDR V4 USB dongle.

<https://github.com/librtlsdr/librtlsdr>

Each version can be built in its respective folder:

fm_demod_gpu_naive, fm_demod_gpu_opt, or fm_demod_gpu_multi.

cd fm_demod_gpu_naive for GPU naive version

Run make to build each version from their folder

make

Run the executable with optional one letter args

./fm_rx_gpu_naive -f 95100000 -o out.wav GPU naive

version

./fm_rx_gpu_multi -c "-400000,-200000,0,200000,400000" -o out multi-channel version

-f allows the user to specify the frequency / station (default is 95.1MHz)

-s allows the user to specify a different demodulated offset than the tuned center

-o allows the user to specify the name of an output .wav file

-r allows the user to set a different sample rate

-t allows the user to specify a time range on the collected audio

-b runs the pipeline on a collected .bin file with raw IQ samples

-c only for the multi-channel version, used to specify offsets of each station as csv.

Or use demo.sh in the root directory to build and run all four versions, including the generic cpu version, then print out a timing comparison.

./demo.sh

Evaluation

The demo script runs each implementation while measuring their execution time and CPU usage. This testbench runs the pipeline ten times over the data contained in the same 60 second capture from the dongle. In the table below, a block is a chunk of data ingested by the

GPU pipeline. This block is collected by the RTL dongle in 64ms, comparing this collection time to the execution time of the pipeline gives the speedup multipliers.

Implementation	Channels	Per-block Timing	Real-time Speedup	CPU utilization
CPU	1	7.58ms	8.4x	99.9%
GPU naive	1	1.46ms	43.9x	16.0%
GPU optimized	1	0.35ms	181.1x	67.1%
GPU multi-channel	5	1.14ms	56.0x	57.6%

The above table shows the results from the demo benchmark test. The top three implementations run on a single frequency channel, while the last one decodes five FM stations at once. The CPU code is about eight times faster than the dongle is collecting IQ samples meaning the CPU implementation already has a bit of headroom before it starts losing samples. The initial GPU version sped this up to 43.9x faster than the real time collection of data, or about five times faster than the CPU. We can also see the CPU utilization drop drastically since all of the processing is now moved to the GPU. The optimized version brings the speedup to 181 times faster than the RTL dongle is collecting samples, or 21 times faster than the CPU version and four times faster than the naive GPU version. We can also see the CPU utilization go up in the optimized version because the host code spends less time waiting for the GPU kernels to finish and more time scheduling future kernels or transferring data. Finally, The multi-channel version decodes five channels and is still faster than the CPU version and the naive GPU version at 56 times faster than real time, 6 times faster than the CPU version, and about 1.5 times faster than the naive GPU version.

Problems

The FM algorithms were the initial difficult portion of this project. I had to brush up on signal processing concepts and familiarize myself with the math. After this, I spent time reading through the RTL dongle documentation and repo to get the device working on the Jetson. Once I had the hardware working and an understanding of the math behind the software, writing the software was not too complicated for the initial CPU and GPU versions. The one issue I had on the naive GPU version was a bit of distortion in the sound initially, which I found to be a GPU warm up issue. These issues were resolved after running the pipeline once before getting actual data.

The project got much more complicated when trying to optimize the kernels and the host interface for maximum performance. For example, I ended up using shared memory on the FM demodulation code since there is overlapping memory usage, but this was complicated by portions of the data needing to be passed between kernel calls. This was fixed by adding a structure that keeps this data between kernel calls and having the first thread load these values. I also ran into an issue when implementing the parallel reduction strategy on the dc removal block. I was forced to break this kernel into two, one that does the parallel reduction summation and one that uses that final sum to normalize and subtract from each element.

The multi-channel implementation was the final leap after the optimized version was verified. For this I had to rework the interface between the host and GPU code. The cuda kernels and C functions mostly stayed the same, but the struct carrying data between kernel calls was updated to include information for each channel. The interface was also updated to use multiple streams to facilitate parallel processing across multiple FM stations. This created some minor synchronization issues that were eventually resolved.

Contributions

Gage Shaddock 100%